# Parallel and Distributed Computing

## Assignment 7

**Instructor:** Dr. Saif Nalband

**Course Code:** UCS645

---

## Problem 1: CUDA Program for Sum of N Integers

### Objective

Develop a CUDA program where different threads perform tasks to compute the sum of the first NNN integers using:

- Iterative approach
- Direct mathematical formula

---

### Approach

#### Iterative Method

- Each thread processes one element
- Uses atomic operations for correct accumulation

#### Formula Method

- Uses formula:

N(N+1)2\frac{N(N+1)}{2}2N(N+1)

- Executed by a single thread

---

### Execution Details

- Input size: N=1024N = 1024N=1024
- Block size: 256 threads
- Grid size: Computed dynamically

---

### Output

- Iterative Sum = 524800
- Formula Sum = 524800

---

### Inference

Both approaches produce identical results, confirming correctness. The iterative approach demonstrates parallel computation, while the formula approach is constant-time.

---

### Conclusion

CUDA enables efficient parallel execution. Mathematical formulations can outperform parallel approaches when complexity is low.

---

# Problem 2: Merge Sort (CPU vs GPU)

### Objective

- Implement merge sort using CPU
- Implement parallel sort using CUDA
- Compare performance

---

### Methodology

#### CPU Implementation

- Recursive merge sort
- Step-wise merging (pipeline-style)

#### GPU Implementation

- Used Thrust library (`thrust::sort`)
- Parallel execution on device

---

### Performance Results

- CPU Time: ~0.000196 seconds
- GPU Time: ~0.000905 seconds
- Speedup: ~0.22×

---

### Inference

For small datasets (N=1000N = 1000N=1000), CPU performs better due to:

- CUDA kernel launch overhead
- Memory transfer latency

---

### Conclusion

GPU acceleration is beneficial for large datasets, while CPU remains efficient for smaller inputs.

---

## Problem 3: Vector Addition and Bandwidth Analysis

### Objective

- Implement vector addition using CUDA
- Use static device memory
- Measure kernel performance
- Analyze memory bandwidth

---

### Implementation Highlights

- Used `__device__` arrays
- Avoided `cudaMalloc`
- Used CUDA events for timing

---

## Bandwidth Analysis

### Theoretical Bandwidth

$$BW = \frac{\text{memoryClockRate} \times \text{memoryBusWidth} \times 2}{8}$$

- Clock Rate: 5.00 GHz
- Bus Width: 256 bits
- Theoretical BW: ~320 GB/s

---

### Measured Bandwidth

$$BW = \frac{\text{Read Bytes + Write Bytes}}{\text{Execution Time}}$$

- Measured BW: ~145–165 GB/s
- Efficiency: ~45%–51%

---

## Profiling Insight

As seen in profiling (page 10–11 of your file), most execution time is spent on:

- Memory transfers
- Not kernel computation

---

## Inference

Measured bandwidth is lower than theoretical due to:

- Memory latency

- Hardware limitations
- Access patterns

---

## Conclusion

CUDA provides high performance but cannot fully reach theoretical bandwidth in real scenarios.